



Variable neighborhood search algorithms for the permutation flowshop scheduling problem with the preventive maintenance

Wahiba Jomaa¹ · Mansour Eddaly^{2,3} · Bassem Jarboui⁴

Received: 5 April 2018 / Revised: 21 April 2019 / Accepted: 4 July 2019 / Published online: 8 July 2019
© Springer-Verlag GmbH Germany, part of Springer Nature 2019

Abstract

This paper addresses to the permutation flowshop scheduling problem while considering the preventive maintenance in the non-resumable case. The criterion to be optimized is the makespan. Two variable neighborhood search algorithms are proposed. In the first algorithm, only one initial solution is generated according to a constructive heuristic. In the second algorithm, a learning process using a probabilistic model is introduced to the variable neighborhood algorithm in order to generate the initial solution. The computational results show the high performance of the proposed algorithms according to the compared approaches. Besides, the change of the initial solution during the optimization procedure may improve the performance of the variable neighborhood search algorithm.

Keywords Permutation flowshop · Preventive maintenance · Variable neighborhood search · Makespan

Mathematics Subject Classification 90B25 · 90B35 · 68W40

✉ Mansour Eddaly
eddalymansour@gmail.com

Wahiba Jomaa
wahiba.jomaa@hotmail.fr

Bassem Jarboui
bassem_jarboui@yahoo.fr

¹ MODILS Laboratory, FSEGS, Université de Sfax, Sfax, Tunisia

² College of Business and Economics, Qassim University, Buraydah, Kingdom of Saudi Arabia

³ ISAEG, Université de Gafsa, Gafsa, Tunisia

⁴ Higher Colleges of Technology, Abu Dhabi, United Arab Emirates

1 Introduction

In most studies related to flow-shop scheduling problem, the common assumption is that the machines are always available for processing jobs. However, this assumption seems to be restrictive from a practical point of view because, after operating a period of time, a machine may stop for a maintenance work. In the stochastic case, the breakdown and the maintenance task are random, so the maintenance is called corrective. Otherwise, the maintenance task can be carried out at predetermined time intervals to keep machine at a high level of operation, it's a preventive maintenance Ying et al. (2010). Whatever it is corrective or preventive, the maintenance is highly recommended in production systems to keep its RMAS: Reliability, Maintainability, Availability and Safety (Al-Turki et al. 2014; Birolini 2010).

The preventive maintenance has proved its efficiency in many manufacturing systems. Such that in the aerospace industry, a stochastic unavailability has high costs and the maintenance has to be planned in advance Birolini (2010). In the literature, the problem of maintenance scheduling has been studied in two different ways. Some authors have considered the scheduling problem separately to the maintenance operations. Other works have considered the maintenance as a constraint of scheduling problem.

The scheduling problem with preventive maintenance (PM) can be considered with two processing modes. The non-resumable mode occurs if the maintenance operation interrupts the processing of a job and the latter has to be restarted from the beginning after the machine becomes available. If the job can be continued later after the machine becomes available again without loss of time or at no cost the problem, it is called "resumable" Ying et al. (2010).

The flowshop scheduling problem with fixed unavailability of machines has been treated by many researchers. Yang et al. (2008) have studied the case of two machines flowshop with separated unavailability constraint and a heuristic algorithm has been proposed to solve the problem. Allaoui et al. (2008) have studied the same problem in the non-resumable case to minimize the makespan. The authors have proposed some properties of the optimal solution and have proved that the complexity order is NP-hard. Xu et al. (2009), for the same objective, have considered the two parallel machines scheduling problem where the unavailability occurs periodically in one machine. The worst case ratio of two well known scheduling heuristics are showed. Xu and Yang (2013), have extended the work of Xu et al. (2009) by establishing mathematical programming model and presenting the average case analysis of the classical heuristics.

For the case of multiple machines, Xu et al. (2008) have considered the parallel machines scheduling with almost periodic maintenance activities in the non-resumable case to minimize the makespan and they have proposed a BFD-LPT (Best Fit Decrease-Least Processing Time) algorithm as the best possible polynomial time $\frac{2T'}{T}$ approximation to solve the problem if $T' = T$ where T and T' denote the minimum and maximum maintenance spacing, respectively. Lee and Chen (2000) for an objective of minimizing the total weighted completion time of

jobs in the case of multiple parallel machines have proposed a branch and bound algorithm and have noted that it can solve medium size problems. Ruiz et al. (2007) have studied the m machines permutation flowshop scheduling problem under the makespan minimization. They have proposed three different policies to determine the period to perform the preventive maintenance tasks and it has been proved that implicitly taking under consideration preventive maintenance operations in sequencing a problem gives better results than solving them separately. The first policy is based on the principle that the PM tasks are processed at fixed predefined time intervals. This policy (planned maintenance) is widely used in industry, where the tasks of preventive maintenance are executed in predefined intervals of time without taking into consideration the best moments of making maintenance tasks. In this policy, the production process can be affected when stochastic breakdowns takes place. The second policy consists on calculating the optimum period for preventive maintenance in order to maximize the machine's availability. In the third policy, the criterion of maintaining a minimum reliability threshold for a given production period t is considered. The reliability of a production machine is the probability that the machine works during a given period of time t . In our work, we consider the second and the third policies as in Ruiz et al. (2007).

Besides, Besbes et al. (2010) have considered the hybrid flowshop scheduling problem with unavailability constraints while studying two variants of preventive maintenance strategies. The first one deals with the fixed starting times of maintenance tasks, whereas in the second variant, maintenance must be performed on given time windows. A genetic algorithm was developed to minimize the makespan criterion. Based on statistic tools, the authors have concluded that the use of the flexible or the fixed variant of maintenance policy depends on the duration of the maintenance task. Aggoune (2004) has addressed the general flowshop scheduling problem with the two variants of maintenance strategies given in Besbes et al. (2010). To solve this problem, the author has proposed a genetic algorithm (GA) and a tabu search algorithm (TS). The GA as well as the TS were developed according the basic framework that is to say using the simple operators. The main conclusion of this work is that the use of time windows is more profitable than fixed maintenance operations. Naderi et al. (2011) have considered the three maintenance policies of Ruiz et al. (2007) to deal with the preventive maintenance in the flexible flowshop scheduling problem. The genetic algorithm and the artificial immune system (AIS) were the solution methods of this problem beyond some well known constructive heuristics.

Although, Ruiz et al. (2007) have invented the framework of the problem in the literature, we do not find the originality of the proposed solving methods. The latter solving algorithms were either elementary or inspired by previous works already proposed for the classical permutation flowshop scheduling problem like the known NEH heuristic of Nawaz et al. (1983), the Simulated Annealing algorithm of Osman and Potts (1989), the Tabu Search algorithm of Widmer and Hertz (1989), the Genetic Algorithm of Ruiz et al. (2006) and the Ant Colony Optimization algorithm of Rajendran and Ziegler (2004).

Benbouzid-Sitayeb et al. (2011) consider the permutation flowshop scheduling problem (PFSP) jointly with the preventive maintenance. Two strategies are employed called sequential and integrated. The sequential strategy is a bi-phase approach where the Preventive maintenance tasks are inserted into the best production sequence obtained so far. The integrated strategy consists on optimizing simultaneously both the production and the maintenance processes using a combination of two criteria. The NEH heuristic, Genetic algorithm and Tabu Search algorithm are used to solve the problem. The computational results show that the genetic algorithm is the best one. Moreover, Benbouzid-Si and Belkaaloul (2014) have investigated the PFSP with preventive maintenance where only one maintenance task is performed on each machine. The objective was to minimize the total earliness/tardiness of all preventive maintenance operations. In this work, the AIS was used for solving the problem. In the first phase, the PFSP is solved without maintenance tasks and then the maintenance operations were inserted. In the literature, this is proven to be not efficient and the simultaneous strategy is the best.

In this paper, we consider the multiple machines permutation flowshop scheduling problem with the preventive maintenance in order to minimize makespan. We aim to solve the considered problem using a Variable Neighborhood Search algorithm (VNS).

The Variable Neighborhood Search (VNS) is a local search based metaheuristic. It was introduced by Mladenović and Hansen (1997) as an efficient approach for solving NP-hard problems mainly the scheduling problems with maintenance (Aloise et al. 2006; Laalaoui and M'Hallah 2016; Ladj et al. 2017; Msakni and Haouari 2018; Pacheco et al. 2018; Todosijević et al. 2016; Yazdani et al. 2017). This algorithm proceeds from an initial solution which is manipulated systematically by local changes of the neighborhood structure. At each iteration, the value of the objective function is improved until a local optimum is found. It is noteworthy that the wide success of the VNS method for a large number of applications is due to its ability to frequently refresh the current solution in the search because of the need of periodical diversification without being stuck in a local optimum or deteriorating the solution's quality Hansen and Mladenović (2001). In its basic version, the VNS started with only one solution which may be the main reason to find 'bad' results if this solution was not good. To overcome this disadvantage in our paper, we propose to introduce a learning effect process based on a probabilistic model to enhance the diversification phase of the VNS algorithm.

The rest of the paper is organized as follows: Sect. 2 describes the problem under consideration. Section 3 presents the way of assigning the maintenance task to the jobs sequence. Section 4 presents the proposed approach. Section 5 describes the experimental results to evaluate the performance of the proposed algorithms. Finally, Sect. 6 presents our conclusions.

2 Problem description

The m -machines permutation flowshop scheduling problem with preventive maintenance can be described as follow: there is a set of n independent non-resumable jobs which are processed on a set of m machines in the same order $1, 2, \dots, m$. All

jobs are available at time zero. Each job j , $j = 1, 2, \dots, n$, requires a processing time in each machine i , $i = 1, 2, \dots, m$. Let p_{ij} and C_{ij} be the processing time and the completion time of the job j on the machine i , respectively. The scheduling of non-resumable jobs means that the jobs cannot be processed if they cannot be finished before the preventive maintenance task is performed. The preventive maintenance tasks are executed with a known processing times, more than one PM task can be planned in one machine. The time units that each machine should exactly processed before a preventive maintenance task is performed is denoted by TPM_i . The processing time of the l th maintenance task on the machine i is denoted by T_{li} . We note that the TPM_i and T_{li} are known and given as input data in advance. The optimization criterion is makespan minimization $C_{max} = \max \{C_{ij}\}$. By using the three field $\alpha|\beta|\gamma$ notation of Graham et al. (1979), the studied problem is denoted by $F_m|n_r - a|C_{max}$ which means minimizing the makespan in the m -machine flowshop scheduling problem under a non-resumable case and a constraint of availability. The problem of scheduling jobs on two machines flowshop with an availability constraint is NP-complete when there is a single interval of non-availability on one machine only Lee (1997). Moreover, Kubiak et al. (2002) proved that the problem becomes NP-hard in the strong sense even if an arbitrary number of holes occur on one machine only. As a result, the problem of this study is NP-hard in the strong sense.

3 The maintenance tasks assignment

In the first heuristic, denoted by H1, in each machine, the accumulated total processing time is calculated considering the processing of a new operation. If this time exceeds the TPM_i , then the process of the new job is necessarily preceded by the process of the maintenance task. This heuristic is proposed by Ruiz et al. (2007).

The computational results of this heuristic show that, several machines sit idle before processing the task maintenance which leads to poor utilization of resources. Therefore, we propose a new heuristic, denoted by H2, consists in assigning the maintenance task in the largest idle time slot before reaching the TPM_i . Let $C_{i[j]}$ be the completion time of the job located in the j th position on the machine i and $I_{i[j]}$ be the idle time slot before the job located in the j th position on the machine i . So, $I_{i[j]} = \max\{0, C_{i-1[j]} - C_{i[j-1]}\}$. This heuristic improves the utilization of resources by minimizing the idle time.

The details of the proposed heuristic H2 are given in Algorithm 1.

Algorithm 1: Heuristic for maintenance tasks assignment

```

Function H2 ( $C_{i[j]}$ );
1  Compute the idle time  $I_{i[j]}$ ;
2   $i = 1$  //first machine;
3  insert the preventive maintenance (PM) tasks with respect to only the TPM constraint;
4  do
5      Update  $I_{i[j]}$ ;
6      Compute the accumulated processing times  $s$  from the latest assigned PM task;
7       $j^*$ : is the position where  $s > TPM$ ;
8      Find the largest  $I_{i[j]}$  until  $j^*$ ;
9      Insert the preventive maintenance (PM) tasks into the largest  $I_{i[j]}$ ;
10      $i++$ ;
    while  $i > m$ ;

```

4 The proposed variable neighborhood search approaches

To solve the problem of scheduling m –machines flowshop with preventive maintenance tasks, two algorithms based on the Variable Neighborhood Search (VNS) are proposed.

4.1 The steps of the proposed algorithms

In this section we discuss the detailed steps of the proposed VNS algorithms.

4.1.1 The original VNS algorithm

Let $f(x)$ be the objective function to be minimized where $x \in X$ and X be a set of feasible solutions. Let N_k , $k = 1, 2, \dots, k_{max}$, be the set of neighborhood structures and $N_k(x)$ is the k th neighborhood of a solution x . The original VNS algorithm of Mladenović and Hansen (1997) performs the following steps until reaching a stopping criterion:

- *Step 1* Generate an initial solution x^0 ;
- *Step 2* Choose a set of neighborhood of x^0 , $N_k(x^0)$, $k = 1, 2, \dots, k_{max}$, set $k = 1$;
- *Step 3* Choose a random solution $x' \in N_k(x^0)$;
- *Step 4* Apply the local search to x' to get x'' ;
- *Step 5* If $f(x'') < f(x^0)$, then $x^0 \leftarrow x''$ and $k = 1$. Otherwise set $k = k + 1$. Return to step 3.

4.1.2 Initial solution

In order to generate the initial solution, we apply NEH algorithm, proposed by Nawaz et al. (1983) for constructing the sequence of jobs without considering the maintenance constraints. The NEH can be described as follows:

- *Step 1* The jobs are sorted with respect to the decreasing order of sums of their processing times.
- *Step 2* Take the first two jobs and evaluate the two possible schedules containing them. The sequence with better objective function value is taken for further consideration.
- *Step 3* Take every remaining job in the permutation given in Step 1 and find the best schedule, by placing it at all possible positions in the sequence of jobs that are already scheduled.

4.1.3 The neighborhood structures

The performance of a VNS algorithm depends on the efficiency of the neighborhood structures with which a systematic switch from one neighborhood to another is met. In our proposed algorithm four neighborhood structures are defined.

The first one, denoted by NS_1 , consists in selecting at random two jobs from the current sequence and swapping their positions (Fig. 1).

In the second one, denoted by NS_2 , we randomly select a job i and insert it in all possible positions in the current sequence (Fig. 2).

The third structure, denoted by NS_3 , divides the current sequence into two parts and swap their positions (Fig. 3).

In the last neighborhood structure, denoted by NS_4 , we randomly choose two cutting points, cutting start ($c - start$) and cutting end ($c - end$), from the current sequence where $(1 < c - start < c - end < n)$, and we translate the set of jobs between the two parts to the beginning of the sequence (Fig. 4).

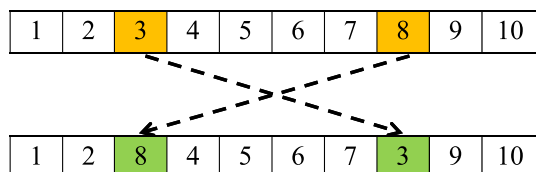
4.1.4 The local search procedures

In our case we use two different local search procedures for each neighborhood structure denoted by LS_1 and LS_2 that performs sequentially at each neighborhood structure.

(a) Interchange every pairs of jobs (LS_1)

This procedure consists to make all possible swaps of pairs of jobs located in positions (i, p) where $1 \leq i \leq p \leq n$, within all parts of solutions. If the obtained solution is better than the best one, the best solution is replaced by this new obtained solution and the number of iterations is reinitialized to 1. Otherwise, we increment the number of iterations.

Fig. 1 Neighborhood structure NS_1



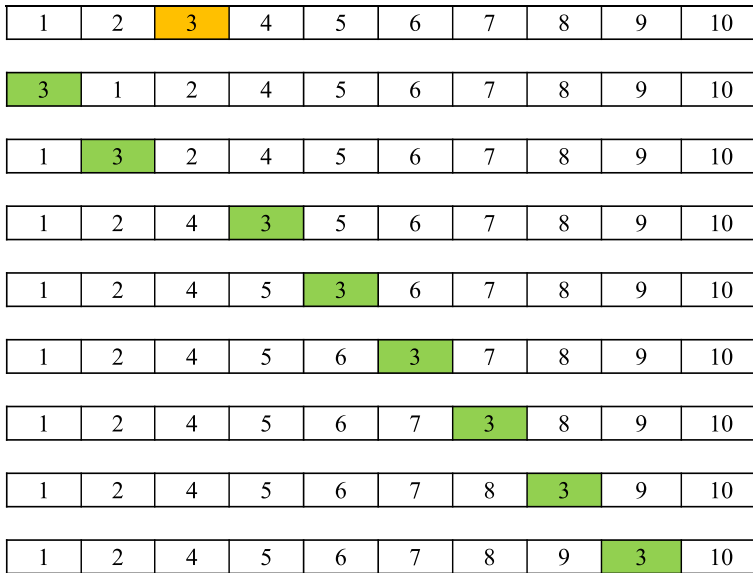


Fig. 2 Neighborhood structure NS2

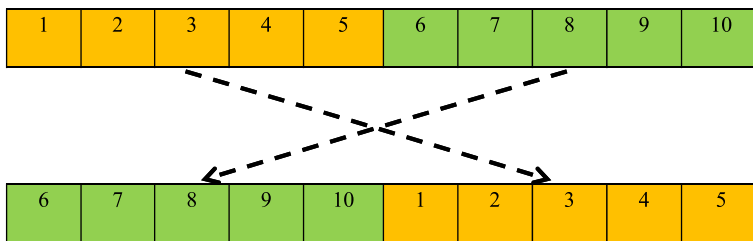


Fig. 3 Neighborhood structure NS3

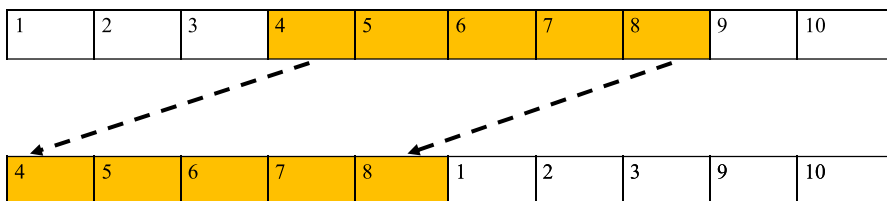


Fig. 4 Neighborhood structure NS4

Algorithm 2: Pseudo code of the swap local search

Input: x_0 is the input solution
Output: x_{best} is the output solution

```

1  $x_{best} = x_0$ ;
2 set  $i = 1$ ;
3 repeat
4   set  $j = i + 1$ ;
5   do
6      $x'$  is the sequence obtained by interchanging the job in position  $i$  and  $j \neq i$ 
       in the sequence  $x_{best}$  ;
7     if  $(C_{max}(x') < C_{max}(x_{best}))$  then
8        $x_{best} = x'$ ;
9        $i = i - 1$ ;
10      break ;
      else
11        $j = j + 1$ ;
    while  $(j \leq n)$ ;
12     $i++$ ;
13    if  $(i > n - 1)$  then
       $i = 1$ ;
until until no possible improvement;
```

(b) Insert all jobs in all possible positions (LS_2)

This procedure consists of all possible insert moves of pairs of jobs located in positions (i, p) where $1 \leq i \leq p \leq n$, within all parts of solutions. If the obtained solution is better than the best one, the best solution is replaced by this new obtained solution and the number of iterations is reinitialized to 1. Otherwise, we increment the number of iterations.

Algorithm 3: Pseudo code of the insert local search

Input: x_0 is the input solution
Output: x_{best} is the output solution

```

1  $x_{best} = x_0$ ;
2 set  $i = 1$ ;
3 repeat
4   set  $j = 1$ ;
5   do
6      $x'$  is the sequence obtained by inserting the job in position  $i$  and  $j \neq i$  in the
       sequence  $x_{best}$  ;
7     if  $(C_{max}(x') < C_{max}(x_{best}))$  then
8        $x_{best} = x'$ ;
9        $i = i - 1$ ;
10      break ;
      else
11        $j = j + 1$ ;
    while  $(j \leq n)$ ;
12     $i++$ ;
13    if  $(i > n - 1)$  then
       $i = 1$ ;
until until no possible improvement;
```

4.2 The proposed algorithms

Based on the VNS, two approaches have been proposed. The first approach optimizes simultaneously the scheduling of jobs and the assignment of the maintenance tasks. At each move, in the production sequence as well as in the neighborhood structure or in the local search procedure, the PM tasks will be assigned and

the evaluation of the problem is performed while taking into account the maintenance constraints. This rule is also applied in the insert moves of the NEH heuristic. This approach is denoted in the following as VNS1.

The efficiency of the VNS algorithm is quite sensitive to the choice of the initial solution as all the other local search based algorithms. In VNS1, the initial solution is generated according to the NEH heuristic. In order to overcome this drawback, we propose to introduce a learning process in the VNS algorithm through a probabilistic model incorporated in an Estimation of Distribution Algorithm (EDA) to generate the initial solution. In what follows, this proposed VNS will be denoted by VNS2. The EDA is a population based algorithm developed by Mühlenbein and Paaß (1996). From a population of P solutions, generally randomly generated, this algorithm selects a subset of M solutions with respect to their fitness using the roulette wheel selection procedure. Next, the EDA uses a probabilistic model learned from a population of solutions. Therefore, a distribution of probability is estimated from the selected candidates of the initial population and then new offspring is generated according to this distribution. Finally, according to the quality of the new solution, it is decided whether it will survive in the next generation.

In our proposition, the used probabilistic model is the one proposed by Jarboui et al. (2009) which can be described as follows:

We denote by η_{ih} the number of times of appearance of job i before or in the position h in the subset of the selected sequences augmented by a given constant δ_1 and by $\mu_{i[h-1]}$ the number of times of appearance of job i immediately after the job in the position $h - 1$ in the subset of the selected sequences augmented by a given δ_2 . The estimated probability to schedule the job i in the position h is given by:

$$\pi_{ih} = \frac{\eta_{ih} \times \mu_{i[h-1]}}{\sum_{r \in \Omega_h} \eta_{ir} \times \mu_{i[h-1]}} \quad (1)$$

The details of the proposed VNS2 are given in Algorithm 1.

Algorithm 4: Steps of the VNS2

```

Function VNS2 ( $k_{max}$ );
1  Generate  $P$  initial solutions randomly ;
2  repeat
3    Select  $M$  candidates using roulette wheel selection;
4    Generate new offspring  $x^0$  using the Eq.(1) ;
5    repeat
6       $x' \leftarrow$  Neighborhood structures( $x^0, k$ ) // NS1, NS2, NS3 NS4;
7       $x'' \leftarrow$  Local search procedures( $x'$ ) // LS1, LS2;
8      Neighborhood Change( $x^0, x'', k$ );
9    until  $k = k_{max}$ ;
until stopping criterion;
```

5 Experimental results

In this section, the proposed VNS1 and VNS2 are tested according to the second policy and the third policy of Ruiz et al. (2007). Moreover, the two algorithms are evaluated using H1 and H2 heuristics for the maintenance task assignment. All algorithms are coded in C++ and implemented on a computer with a quad core 2.4 GHz processor and a 4 GB RAM. The parameters of the algorithms are set experimentally: the size of the population $P = 20$, the subset of the selected solutions $M = 4$ and the stopping criterion is set at a fixed elapsed time $n \times \frac{m}{2} \times 0.025$ seconds as in Ruiz et al. (2007).

In order to evaluate the effectiveness of the proposed algorithms, the set of 3600 instances generated by Ruiz et al. (2007) are used. The number of jobs n varies from 20 to 500 and the number of machines m from 5 to 20.

The performance measure used for the evaluation is the Relative Percentage Deviation $RPD = \frac{Alg_{sol} - Min_{sol}}{Min_{sol}} \times 100$. For VNS1 and VNS2 the minimum, the average and the maximum RPD are given since every algorithm has been executed five times.

The Tables 1 and 2 display the results obtained by VNS1 and VNS2 for both policies (II and III). We recall that the policy II aims to maximize the machine's availability whereas the policy III aims to maintain a minimum level of reliability threshold for a given production period. It is clear that the results of H2 heuristic are better than those of H1 heuristic. So, reducing the idle times of each machine leads to an improvement in the total makespan. Moreover, based on H2 heuristic, VNS2 algorithm outperforms VNS1 in all deviation measures (min, avg and max). Thus, the introduction of the learning process contributes, significantly, to ameliorate the quality of the solutions. At the CPU time level (Tables 3, 4), as expected, the population based algorithm VNS2 appears to be much time consuming than VNS1. It can be explained by the fact that the initial solution of VNS2 is based on a whole population and a probabilistic model whereas it is simply generated according to a priority rule for VNS1.

In Tables 5 and 6, our best proposed approach (VNS2 based on H2 heuristic) is evaluated according to the approaches of the literature. The compared approaches consists of six algorithms already proposed for the classical permutation flowshop scheduling problem and have been adopted by Ruiz et al. (2007) to solve the problem with maintenance constraints. These approaches include the NEH heuristic of Nawaz et al. (1983), the Simulated Annealing algorithm of Osman and Potts (1989) (SA-OP), the Tabu Search algorithm of Widmer and Hertz (1989) (SPIRIT), the Genetic Algorithm of Ruiz et al. (2006) (GARMA), Ant Colony Optimization algorithm of Rajendran and Ziegler (2004) and a random solution (RANDOM) to designate the worst case. Concerning the policy III, VNS2 gives a non positive average results in terms of all minimum deviations (-0.25) and the average results of VNS2 are also non positive for both minimum (-0.47) and average (-0.14) deviations.

To sum up, at first, it is shown that trying to assign the maintenance task into the idle time slots of the machines may enhance the resource utilization while

Table 2 Computational experiments of the proposed approaches according to the third policy based on H1 and H2 heuristics for maintenance activities assignment

Instances	H1 heuristic						H2 heuristic					
	VNS1			VNS2			VNS1			VNS2		
	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max
20 × 5	-0.2	0.48	0.55	0.33	0.45	0.51	-0.11	-0.01	0.38	-0.27	-0.21	-0.15
20 × 10	-1.62	-0.39	-0.09	-0.80	-0.51	-0.25	-1.26	-0.87	-0.69	-1.78	-1.40	-1.09
20 × 15	-2	-0.77	-0.42	-1.25	-0.89	-0.59	-1.57	-1.13	-1.09	-2.14	-1.69	-1.32
20 × 20	-1.93	-0.71	-0.31	-1.24	-0.88	-0.56	-1.5	-1.09	-1.05	-2.12	-1.66	-1.30
50 × 5	0.01	0.25	0.31	0.16	0.24	0.29	0.08	0.16	0.18	-0.01	0.03	0.07
50 × 10	-0.11	0.2	0.56	-0.23	0.14	0.47	-0.15	0.31	0.71	-0.44	-0.05	0.27
50 × 15	-0.55	-0.29	0.26	-0.95	-0.36	0.14	-0.84	0.1	0.69	-1.16	-0.47	0.03
50 × 20	-1.05	-0.58	-0.03	-1.29	-0.66	-0.17	-1.15	-0.37	0.25	-1.50	-0.81	-0.30
100 × 5	0.03	0.22	0.29	0.12	0.20	0.23	0.08	0.13	0.16	0.00	0.03	0.06
100 × 10	0.26	0.47	0.81	0.10	0.42	0.71	0.18	0.58	0.89	-0.05	0.30	0.58
100 × 15	0.34	0.54	0.91	-0.32	0.27	0.77	-0.19	0.99	1.32	-0.35	0.23	0.65
100 × 20	-0.05	0.26	0.58	-0.86	-0.20	0.37	-0.63	0.72	1.05	-0.89	-0.22	0.27
200 × 5	0.14	0.2	0.28	0.13	0.18	0.24	0.14	0.17	0.2	0.08	0.12	0.14
200 × 10	0.45	0.59	0.75	0.14	0.40	0.67	0.19	0.63	0.65	0.12	0.33	0.47
200 × 15	0.57	1.09	1.15	0.03	0.51	0.97	0.12	1.16	1.16	0.03	0.48	0.82
200 × 20	0.45	0.66	1.07	-0.34	0.29	0.86	-0.14	0.7	0.73	-0.35	0.18	0.49
500 × 5	0.22	0.25	0.34	0.12	0.21	0.30	0.17	0.22	0.22	0.08	0.16	0.20
500 × 10	0.55	0.56	0.76	0.34	0.55	0.74	0.37	0.55	0.55	0.32	0.46	0.52
500 × 15	0.78	0.86	1.16	0.54	0.83	1.12	0.58	0.78	0.79	0.51	0.69	0.75
500 × 20	0.82	0.91	1.33	0.47	0.86	1.22	0.54	0.82	0.83	0.44	0.70	0.80
Average	-0.14	0.24	0.51	-0.24	0.10	0.40	-0.25	0.23	0.40	-0.47	-0.14	0.10

Table 3 CPU times based on H1 heuristic for maintenance activities assignment

Instances	Policy II				Policy III						
	VNS1		VNS2		VNS1		VNS2				
	Min	Avg	Max	Min	Avg	Max	Min	Avg	Max		
20 × 5	0.04	0.18	0.36	0.99	1.00	1.00	0.02	0.11	0.24	0.99	1.00
20 × 10	0.34	1.13	2.00	2.49	2.50	2.50	0.29	0.99	1.81	2.49	2.50
20 × 15	0.48	1.61	2.85	3.48	3.50	3.50	0.50	1.59	2.74	3.48	3.50
20 × 20	0.74	2.34	4.05	4.97	4.99	5.00	0.70	2.29	3.94	4.98	5.01
50 × 5	0.25	0.65	1.17	2.43	2.49	2.52	0.19	0.47	0.89	2.43	2.52
50 × 10	1.03	2.88	4.97	6.11	6.22	6.29	0.96	2.83	4.90	6.12	6.29
50 × 15	1.22	4.24	7.31	8.50	8.71	8.82	1.78	4.47	7.57	8.55	8.82
50 × 20	2.48	6.45	10.68	12.18	12.44	12.58	2.61	6.45	10.69	12.15	12.58
100 × 5	1.16	2.46	4.68	4.49	4.90	5.18	0.98	1.73	2.80	4.50	5.17
100 × 10	4.04	8.06	11.46	11.36	12.33	12.84	4.20	7.83	11.32	11.54	12.89
100 × 15	7.62	13.12	17.16	15.84	17.27	18.03	7.68	12.98	16.98	16.12	18.07
100 × 20	11.38	18.77	24.43	22.79	24.67	25.66	12.00	19.43	24.58	22.91	25.73
200 × 5	5.91	7.89	9.38	6.80	9.05	10.55	6.96	8.52	9.75	6.47	11.00
200 × 10	17.01	22.04	25.25	19.40	24.15	27.73	17.58	22.15	24.97	18.65	27.91
200 × 15	28.08	33.18	35.92	28.85	35.16	39.21	26.63	32.68	35.82	28.89	39.96
200 × 20	40.84	47.71	51.09	42.31	50.17	55.53	41.19	48.01	51.37	42.46	56.32
500 × 5	26.45	30.76	36.47	27.57	35.44	45.72	25.68	27.63	30.77	28.83	48.30
500 × 10	64.25	69.33	75.39	63.67	72.24	89.49	64.16	68.22	72.05	63.39	88.87
500 × 15	90.25	99.02	108.95	90.02	97.66	111.87	91.73	103.49	114.58	91.40	112.63
500 × 20	127.42	136.36	148.58	127.77	137.25	151.22	129.14	140.77	155.34	126.84	153.56
Average	21.5	25.4	29.1	25.10	28.11	31.76	21.75	25.63	29.16	25.16	32.13

Instances	Policy II				Policy III							
	VNS1			VNS2	VNS1			VNS2				
	Min	Avg	Max	Min	Min	Avg	Max	Min	Avg	Max		
20 × 5	0.07	0.23	0.43	0.98	0.99	1.00	0.04	0.13	0.25	0.98	0.99	1.00
20 × 10	0.48	1.25	2.02	2.46	2.48	2.50	0.38	1.05	1.84	2.46	2.48	2.50
20 × 15	0.76	1.86	2.92	3.44	3.48	3.50	0.71	1.75	2.84	3.44	3.48	3.50
20 × 20	1.07	2.62	4.15	4.91	4.96	5.00	1.03	2.66	4.25	4.91	4.96	5.00
50 × 5	0.37	0.81	1.35	2.25	2.41	2.50	0.29	0.64	1.08	2.31	2.42	2.50
50 × 10	1.69	3.59	5.45	5.54	5.99	6.25	1.79	3.44	5.10	5.69	6.04	6.25
50 × 15	3.11	5.64	8.03	7.49	8.30	8.75	3.04	5.57	7.93	7.76	8.39	8.74
50 × 20	4.20	7.95	11.51	10.60	11.81	12.50	3.79	7.68	11.26	10.96	11.92	12.50
100 × 5	1.88	2.71	3.68	3.18	4.27	5.01	1.61	2.26	3.02	3.51	4.40	4.99
100 × 10	5.97	8.60	11.22	8.55	11.05	12.55	4.64	7.86	11.04	8.49	11.01	12.51
100 × 15	9.52	12.69	16.13	10.91	15.13	17.61	8.30	12.12	15.90	10.35	14.76	17.47
100 × 20	13.99	18.55	23.04	15.54	21.79	25.22	12.37	17.43	22.77	14.37	20.74	24.82
200 × 5	8.62	8.93	9.27	7.96	9.68	11.20	6.85	7.43	8.03	6.85	8.87	10.21
200 × 10	22.26	23.16	24.19	21.43	24.90	27.63	20.23	21.20	22.59	18.97	23.24	25.64
200 × 15	32.89	33.66	34.55	33.07	35.72	38.27	35.74	35.98	36.20	30.78	34.38	36.42
200 × 20	48.33	49.02	49.98	47.88	50.99	54.31	47.33	48.20	48.99	45.40	49.49	51.88
500 × 5	29.48	29.65	29.86	26.78	30.78	35.58	34.67	34.76	34.90	29.43	35.53	40.16
500 × 10	75.61	75.63	75.65	64.32	70.48	78.18	82.96	84.17	85.32	67.08	78.85	91.03
500 × 15	105.10	105.13	105.18	89.36	95.91	103.95	103.86	104.76	105.44	88.29	93.41	103.42
500 × 20	146.42	146.45	146.49	126.25	133.84	145.89	141.13	143.20	145.11	126.99	137.33	151.90
Average	25.59	26.91	28.26	24.64	27.25	29.87	25.54	27.11	28.69	24.45	27.63	30.62

Table 5 Computational experiments of the best proposed approaches according to the second policy against the approaches of the literature

Instances	RANDOM	NEH	SA-OP	SPIRIT	GARMA	PACO	VNS2 based H2 heuristic		
							Min	Avg	Max
20 × 5	5.77	2.43	0.87	3.16	0.15	0.65	0.43	0.59	0.71
20 × 10	9.36	2.25	1.38	2.85	0.26	0.21	− 1.10	− 0.65	− 0.29
20 × 15	5.61	0.59	0.50	2.41	0.08	0.05	− 1.61	− 1.08	− 0.67
20 × 20	6.15	2.51	0.90	5.02	0.12	0.79	− 2.23	− 1.77	− 1.38
50 × 5	7.47	2.18	1.05	2.42	0.18	0.28	0.73	0.87	1.01
50 × 10	8.69	1.76	1.14	2.46	0.16	0.10	0.40	0.94	1.28
50 × 15	5.03	1.46	0.64	4.98	0.07	0.26	− 0.77	− 0.08	0.45
50 × 20	7.75	4.19	1.66	3.18	0.17	0.93	− 1.70	− 0.98	− 0.47
100 × 5	6.83	1.89	0.97	1.83	0.22	0.08	0.88	1.06	1.20
100 × 10	6.72	1.03	0.77	4.81	0.08	0.12	0.59	1.07	1.36
100 × 15	5.74	4.27	1.69	3.01	0.18	1.12	− 0.39	0.25	0.76
100 × 20	8.72	4.32	1.79	2.42	0.29	0.48	− 1.31	− 0.53	0.06
200 × 5	5.43	1.40	0.61	2.37	0.09	0.14	1.35	1.51	1.61
200 × 10	6.09	4.03	1.58	5.28	0.11	1.29	0.78	1.14	1.40
200 × 15	7.14	4.19	1.25	2.29	0.22	0.64	− 0.30	0.24	0.63
200 × 20	7.64	3.65	1.14	2.05	0.14	1.67	− 0.76	− 0.13	0.32
500 × 5	4.84	2.54	0.75	5.01	0.06	0.71	0.69	0.84	0.96
500 × 10	7.58	6.01	2.02	2.98	0.36	1.43	0.47	0.67	0.82
500 × 15	6.48	3.32	1.01	1.38	0.22	0.88	0.22	0.53	0.74
500 × 20	6.26	2.63	0.85	4.80	0.03	1.21	− 0.16	0.21	0.44
Average	6.77	2.83	1.13	3.24	0.16	0.65	− 0.19	0.23	0.55

comparing the results of H1 and H2 heuristics. At second, the introduction of the learning process using the EDA algorithm have improved the quality of the solutions. This is another argument for the sensitivity of the VNS algorithm to the starting point.

6 Conclusions

In this paper the problem of scheduling with preventive maintenance planning in the case of multiple machines flowshop in order to minimize the makespan is studied. Two variable neighborhood search algorithms are developed. The first one starts from only one initial solution built by a constructive heuristic. In the second algorithm, a learning process based on a probabilistic model using an EDA is introduced to generate the initial solution. The obtained results indicates that the latter provides better results according to the previous algorithms from the literature.

Table 6 Computational experiments of the best proposed approaches according to the third policy against the approaches of the literature

Instances	RANDOM	NEH	SA-OP	SPIRIT	GARMA	PACO	VNS2 based H2 heuristic		
							Min	Avg	Max
20 × 5	8.93	8.57	6.02	7.92	5.71	0.19	-0.27	-0.21	-0.15
20 × 10	8.64	7.41	3.74	6.14	2.57	0.56	-1.78	-1.40	-1.09
20 × 15	7.80	6.70	3.31	5.26	1.82	1.06	-2.14	-1.69	-1.32
20 × 20	6.86	5.72	2.81	4.52	1.43	0.88	-2.12	-1.66	-1.30
50 × 5	15.10	11.91	11.18	12.27	10.83	0.05	-0.01	0.03	0.07
50 × 10	11.65	6.67	3.77	5.11	2.54	0.26	-0.44	-0.05	0.27
50 × 15	11.43	6.72	3.40	4.21	1.39	0.53	-1.16	-0.47	0.03
50 × 20	10.52	5.77	3.10	3.67	1.07	0.55	-1.50	-0.81	-0.30
100 × 5	15.54	12.13	11.85	12.62	11.71	0.03	0.00	0.03	0.06
100 × 10	10.42	4.36	2.71	3.94	2.09	0.16	-0.05	0.30	0.58
100 × 15	11.23	5.15	2.84	3.82	1.76	0.29	-0.35	0.23	0.65
100 × 20	10.84	5.06	2.85	3.29	1.32	0.35	-0.89	-0.22	0.27
200 × 5	11.90	8.73	8.88	10.71	8.48	0.04	0.08	0.12	0.14
200 × 10	8.23	2.12	1.85	5.34	1.09	0.12	0.12	0.33	0.47
200 × 15	9.49	2.92	1.87	6.01	0.89	0.25	0.03	0.48	0.82
200 × 20	9.67	3.38	2.06	6.05	1.01	0.40	-0.35	0.18	0.49
500 × 5	3.20	0.59	1.08	3.97	0.58	0.01	0.08	0.16	0.20
500 × 10	5.77	0.38	0.93	7.49	0.24	0.01	0.32	0.46	0.52
500 × 15	7.09	0.74	1.12	9.22	0.33	0.10	0.51	0.69	0.75
500 × 20	7.35	1.17	1.32	9.50	0.53	0.09	0.44	0.70	0.80
Average	9.58	5.31	3.83	6.55	2.87	0.30	-0.47	-0.14	0.10

References

- Aggoune R (2004) Minimizing the makespan for the flow shop scheduling problem with availability constraints. *Eur J Op Res* 153(3):534–543
- Al-Turki O, Ayar T, Yilbas B-S, Sahin A-Z (2014) Integrated maintenance planning in manufacturing systems. Springer, Berlin
- Allaoui H, Lamouri S, Artiba A, Aghezzaf E (2008) Simultaneously scheduling n jobs and the preventive maintenance on the two-machine flow shop to minimize the makespan. *Int J Prod Econ* 112(1):161–167
- Aloise DJ, Aloise D, Rocha CTM, Ribeiro CC, Ribeiro Filho JC, Moura LSS (2006) Scheduling workover rigs for onshore oil production. *Discret Appl Math* 154(5):695–702 (**IV ALIO/EURO Workshop on Applied Combinatorial Optimization**)
- Benbouzid-Si Tayeb F, Belkaaloul W (2014) Towards an artificial immune system for scheduling jobs and preventive maintenance operations in flowshop problems. In 23rd IEEE international symposium on industrial electronics, ISIE 2014, Istanbul, Turkey, June 1-4, 2014, p 1065–1070
- Benbouzid-Sitayeb F, Guebli S-A, Bessadi Y, Varnier C, Zerhouni N (2011) Joint scheduling of jobs and preventive maintenance operations in the flowshop sequencing problem: a resolution with sequential and integrated strategies. *Int J Manuf Res* 6(1):30–48
- Besbes W, Teghem J, Loukil T (2010) Scheduling hybrid flow shop problem with non-fixed availability constraints. *Eur J Ind Eng* 4(10):413–433
- Birolini A (2010) Reliability engineering: theory and practice. Springer, Berlin
- Graham R L, Lawler E L, Lenstra J K, Kan A H G Rinnooy (1979) Optimization and approximation in deterministic sequencing and scheduling: a survey. *Ann Discret Math* 5(2):287–326

- Hansen P, Mladenović N (2001) Variable neighborhood search: principles and applications. *Eur J Op Res* 130(3):449–467
- Jarboui B, Eddaly M, Siarry P (2009) An estimation of distribution algorithm for minimizing the total flowtime in permutation flowshop scheduling problems. *Comput Op Res* 36:2638–2646
- Kubiak W, Błażewicz J, Formanowicz P, Breit J, Schmidt G (2002) Two-machine flow shops with limited machine availability. *Eur J Op Res* 136(3):528–540
- Laalaoui Y, M'Hallah R (2016) A binary multiple knapsack model for single machine scheduling with machine unavailability. *Comput Op Res* 72:71–82
- Ladj A, Tayeb F Benbouzid-Si, Varnier C, Dridi A-A, Selmane N (2017) A hybrid of variable neighborhood search and fuzzy logic for the permutation flowshop scheduling problem with predictive maintenance. *Procedia Comput Sci* 112:663–672
- Lee C-Y (1997) Minimizing the makespan in the two-machine flowshop scheduling problem with an availability constraint. *Op Res Lett* 20(3):129–139
- Lee C-Y, Chen Z-L (2000) Scheduling jobs and maintenance activities on parallel machines. *Nav Res Logist (NRL)* 47(2):145–165
- Mladenović N, Hansen P (1997) Variable neighborhood search. *Comput Op Res* 24(11):1097–1100
- Msakni MK, Haouari M (2018) Short-term planning of liquefied natural gas deliveries. *Transp Res Part C Emerg Technol* 90:393–410
- Mühlenbein H, Paaß G (1996) From recombination of genes to the estimation of distributions i. binary parameters. In: Voigt H-M, Ebeling W, Rechenberg I, Schwefel H-P (eds) *Parallel problem solving from nature—PPSN IV*. Springer, Berlin, pp 178–187
- Naderi B, Zandieh M, Aminnayeri M (2011) Incorporating periodic preventive maintenance into flexible flowshop scheduling problems. *Appl Soft Comput* 11(2):2094–2101
- Nawaz M, Ensor E-E, Ham I (1983) A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. *Omega* 11(1):91–95
- Osman I-H, Potts C-N (1989) Simulated annealing for permutation flow-shop scheduling. *Omega* 17(6):551–557
- Pacheco J, Porras S, Casado S, Baruaque B (2018) Variable neighborhood search with memory for a single-machine scheduling problem with periodic maintenance and sequence-dependent set-up times. *Knowl-Based Syst* 145:236–249
- Rajendran C, Ziegler H (2004) Ant-colony algorithms for permutation flowshop scheduling to minimize makespan/total flowtime of jobs. *Eur J Op Res* 155(2):426–438
- Ruiz R, Garcia-Diaz J-C, Maroto C (2007) Considering scheduling and preventive maintenance in the flowshop sequencing problem. *Comput Op Res* 34(11):3314–3330
- Ruiz R, Maroto C, Alcaraz C (2006) Two new robust genetic algorithms for the flowshop scheduling problem. *Omega* 34(5):461–476
- Todosijević R, Benmansour R, Hanafi S, Mladenović N, Artiba A (2016) Nested general variable neighborhood search for the periodic maintenance problem. *Eur J Op Res* 252(2):385–396
- Widmer M, Hertz A (1989) A new heuristic method for the flow shop sequencing problem. *Eur J Op Res* 41(2):186–193
- Xu D, Cheng Z, Yin Y, Li H (2009) Makespan minimization for two parallel machines scheduling with a periodic availability constraint. *Comput Op Res* 36(6):1809–1812
- Xu D, Sun K, Li H (2008) Parallel machine scheduling with almost periodic maintenance and non-preemptive jobs to minimize makespan. *Comput Op Res* 35(4):1344–1349
- Xu D, Yang D-L (2013) Makespan minimization for two parallel machines scheduling with a periodic availability constraint: mathematical programming model, average-case analysis, and anomalies. *Appl Math Model* 37(14):7561–7567
- Yang D-L, Hsu C-J, Kuo W-H (2008) A two-machine flowshop scheduling problem with a separated maintenance constraint. *Comput Op Res* 35(3):876–883 (**Part Special Issue: New Trends in Locational Analysis**)
- Yazdani M, Khalili S Mohammad, Babagolzadeh M, Jolai F (2017) A single-machine scheduling problem with multiple unavailability constraints: a mathematical model and an enhanced variable neighborhood search approach. *J Comput Des Eng* 4(1):46–59
- Ying M, Chengbin C, Chunrong Z (2010) A survey of scheduling with deterministic machine availability constraints. *Comput Ind Eng* 58(2):199–211 (**Scheduling in Healthcare and Industrial Systems**)